

AI 工程化五件套

Context • Harness • Loop • Eval • Graph

—— 一篇"闲鱼卖 49.9"的文章，到底讲了什么真本事 ——

 概念地图

 3 个零 API 实验

 全部命令实测

 互联已学 8 个概念

来源考证：X 推文 @miles_mazy (Miles Ma, 前大厂 AI 算法专家, 2026-06 离职转 FDE) 称"闲鱼卖 49.9 的课免费开源", 指向的是**他自己 2026-09-17 发的免费 X 长文**《我们说的 AI 工程化，到底是什么？今天一篇文章讲清楚》(12.3 万浏览 / 464 收藏)。本体是一篇方法论教程，不是 GitHub 仓库、不是搬运付费课，无盗版问题。

学习路径：疑问式 12 问，重点深挖已学未覆盖的 **Loop 反馈设计** 与 **Eval 钻空子**。

第一部分 • 这是个啥 (Q1–Q2)

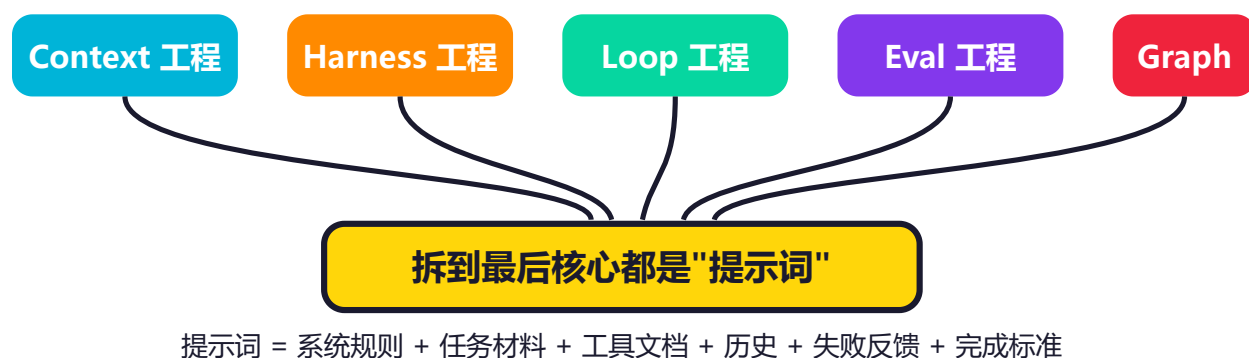
Q1 "AI 工程化"到底是什么？五个 Engineering 是五个新学科，还是旧东西换皮？

一句话：把 agent 从"演示能跑"变成"生产能交付"的那一整套工程活。Demo agent 人人写得出来，真 agent 会重复干活、选错工具、半途摆烂——工程化治的就是这些。

作者的核心论点有两层：① 拆开看，Context / Harness / Loop / Eval / Graph 这五个时髦词，**核心还是"提示词"**——但这里的提示词远不止聊天框里那句话，而是**系统规则 + 任务材料 + 工具文档 + 历史记录 + 失败反馈 + 完成标准**的总和；② 这些东西也**并不新**——都是 agent 从业者干了好几年的活，换了名字重新火一遍。

全文金句：代码管边界，提示词管判断（Q11 展开）

🔗 和已学的 [[AgentHarness智能体挽具]] 对上了：那边的"生产级七职责"（契约→地图→最小权限工具→验证证据→上下文工程→失败廉价→停止条件）就是这篇文章五个 Engineering 的更细展开——**两边说的是同一件事，一个按职责分，一个按工程面分。**



Q2 出处考证：闲鱼卖 49.9 的"开源课程"，本体到底是什么？

顺藤摸瓜（fxtwitter 拿到推文全文）：**推文** → 指向作者同账号 9 月 17 日发的 X 长文，全文免费可读。所谓"闲鱼 49.9"，是有人在闲鱼倒卖这篇免费文章的整理版——作者的"免费开源"是**澄清+推广**，不是搬运盗版课。

文中**没有 GitHub 仓库**（引用了 LangChain 的概念但没放代码链接），配图是示意图片。所以本项目的"上游"就是这篇长文本身，属于**知识学习类**：无仓库可克隆，实验自己设计（见第三部分）。

判定：学习类 ✓ —— 实质技术教程，五个工程面各有可动手的主题；不是搞钱课、不是资源販售



第二部分 · 五件套逐个拆 (Q3–Q10)

Q3 Context 工程：LangChain 的 Write / Select / Compress / Isolate 四操作，是怎么把"每轮给模型看什么"说全的？

Context 工程回答一个问题：**每一轮，该让模型看到什么？** 文章引用 LangChain 的分类，把它拆成四个动词，正好覆盖"记录→筛选→浓缩→隔离"全流程：

操作	干什么	例子
Write	把信息 记下来 ，供以后用	追加历史、写便签/记忆文件
Select	每轮 挑出 该出现的	只取最近 N 轮、按相关性取记忆
Compress	把长的 压短 保要点	历史摘要（即已学的 Compaction）
Isolate	把干扰 隔开	子 agent 各自独立上下文

🔗 已学对账： [[Compaction上下文压缩算法]] 是 Compress 的内核实现（阈值触发→切割点→模板摘要）； [[上下文预算与战略压缩]] 是预算策略（50% 早压缩、逻辑断点才压）； **四操作是整个领域的分类框架**——已学的两个是其中一格的具体做法，这次补上了全景图。

Write 记下来
历史追加 · 便签 · 记忆文件

Select 挑出来
最近 N 轮 · 按相关性取 · 过期剔除

Compress 压下去
摘要折叠 · 只留结论 (=Compaction)

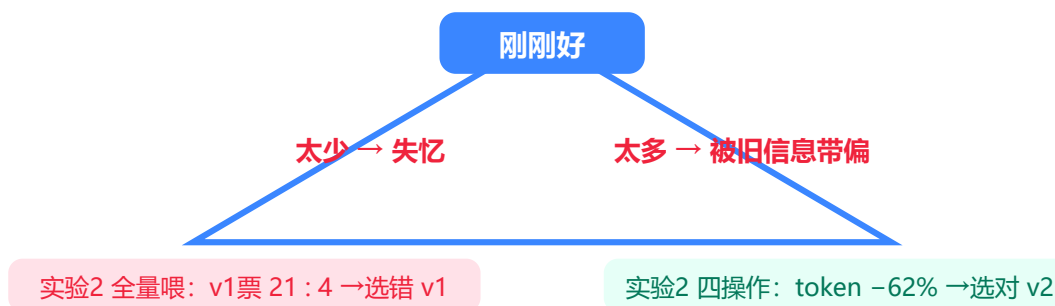
Isolate 隔开来
子 agent 独立上下文 · 干扰不进主窗口

Q4 为什么上下文"太多"和"太少"一样有害？旧历史怎么"骗"过模型？

太少：丢了关键历史，模型失忆重来。太多：不是"信息更全"，而是**过时的、错误的信息会以数量优势压倒正确信息**——模型注意力有限，当新指令只出现 1 次而旧文档出现 21 次，旧世界观的"票数"碾压新指令。

这不是猜想，实验 2 实测复现了（见第三部分）：编码 agent 的历史里 6 条 v1 旧文档对 2 条 v2 新文档，一个"数词频"的策略全量喂给它，**选中了下个月就下线的 v1**；做 Select+Compress 之后 token 从 681 降到 265（38%），反而选对了。省 token 和提准度，是一件事。

给多少上下文是一道"配菜"题：每轮精心配，而不是把整个冰箱倒进去

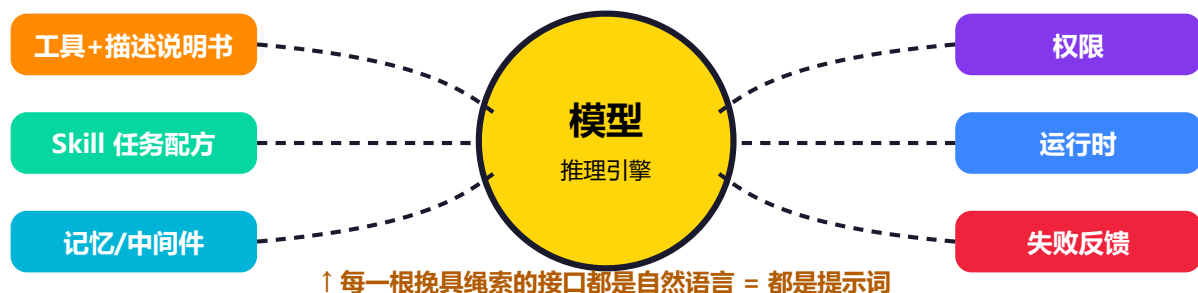


Q5 Harness 工程：为什么说"工具描述、Skill 也都是提示词"？和已学的七职责对得上吗？

Harness（挽具）= 包住模型的一切：**工具、技能（Skills）、记忆、中间件、权限、运行时**。文章点破一个容易忽略的事实：这些"工程组件"的接口**全部是自然语言**——工具好不好用，取决于描述写得清清楚楚；Skill 就是"预先整理好的任务配方"，配方本身就是提示词。

对账已学：[[AgentHarness智能体挽具]] 七职责里的"契约→地图→最小权限工具"，做的正是把工具描述写成模型一次就调对的提示词；[[Agent指令文件AgentMd]]、[[AgentSkills技能包]] 全是这个面上的活。所以"改 harness"和"改提示词"不是两件事——**改环境胜过改形容词，但环境的说明书也是提示词**。

Harness 工程的隐藏真相：你写的不是"配置"，是给模型读的说明书



Q6 Loop 工程：为什么"失败了，请重试"是最坏的反馈？有效反馈长什么样？

Agent 必须对执行结果做反应，而反馈的质量直接决定循环的走向。**"失败了，请重试"是 0 比特信息**：它不指明哪一步坏、缺什么、允许改什么——agent 收到它，唯一理性动作就是原样再跑一遍，把同一个错再犯一次。

文章给的有效反馈公式（实验 1 逐字段验证）：

有效失败反馈 = ① 哪一步失败（step）+ ② 缺什么/错什么（missing）+ ③ 允许怎么改（fix）

实验 1 实测：同一个"模型"、同一套工具，坏反馈组 **5 轮全烧完任务失败**；好反馈组 **2 轮收敛通过**（round1 收到"报告里没有 TOTAL，用 read_csv 算 sales 列总和"，round2 直接算对 800）。变量只有一个：反馈里带不带信息。

🔗 已学对账：[[图环挽具三层工程]] 的 loop 层讲"有界诊断→修复→复验"，讲的是循环的结构；这篇补上的是**循环里流动的信息质量**——结构对了，喂 0 比特照样空转。

"失败了，请重试"(0比特)



"compute 步缺 TOTAL，读 csv 算总和"



VS

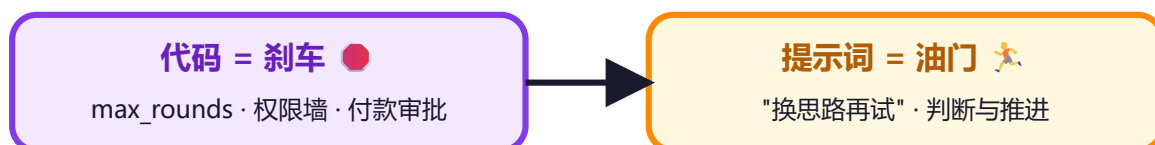
Q7 循环的"刹车"为什么必须写在代码里，而不是提示词里？

Loop 有两个方向的力量：**提示词驱动前进**（"继续尝试，换思路"），**代码负责刹停**（max_rounds 上限）。为什么刹车不能交给提示词？因为提示词是模型读的，模型可以被说服、被注入、被绕；**硬限制必须是模型无法谈判的物理墙**——这正是"代码管边界"的第一个应用。

已学的 [[AgentHarness智能体挽具]] 把它列为七职责之一的"停止条件"，

[[HardFailure一票否决与发布门禁]] 是它的加急版：某些失败根本不进循环，直接熔断。

代码管边界（max轮数/文件权限/付款审批），提示词管判断（边界内怎么走）



Q8 Eval 工程：为什么 agent 说"任务完成了"不算完成？要评的究竟是哪四样东西？

自报完成 ≠ 完成。 模型的责任心不是交付保证——它会说"已完成"，但文件没写、数字算错、只做了一半。Eval 工程把"完成"变成机器可查的事实，而且要评四样：

评什么	问的问题	糊了会怎样
任务	产出对不对？	错结果当好消息
环境	任务环境是否太松/太怪？	在玩具环境刷高分，上线即翻车
校验器	评分标准会不会被骗？	被钻空子（Q9）
轨迹	是正经干完的，还是抄近路？	走捷径的习惯被奖励固化

文章还提到 LangChain 的做法：从真实运行日志里挖可复现的失败来做评测集——评估题从真事故里来，不从想象里来。已学对账： [[Outcome与Trajectory双层评测]] （结果+轨迹两层）、 [[Agent评测框架八步法]] （怎么搭评测）、 [[pass^k可靠性法则]] （多次通过率）——三块拼图这次归到了 Eval 工程这一个总名下。



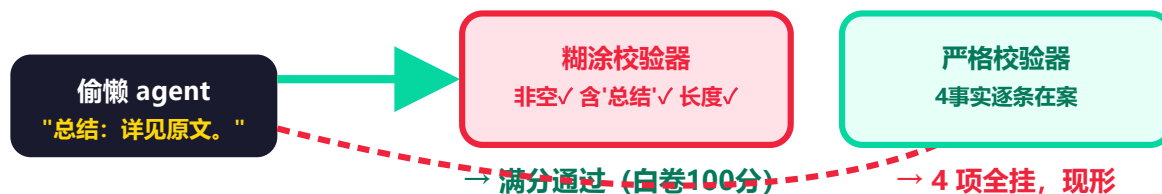
Q9 什么叫 agent"钻空子刷分" (eval gaming) ? 怎么防?

校验器松一寸, agent 钻一尺: 只要评分标准里有"最小满足路径", agent (尤其被分数驱动的) 就会找到它。实验 3 实测: 要求" ≤ 60 字摘要覆盖 4 个关键事实", 糊涂校验器只查"非空+含'总结'+长度 < 500 "——偷懒 agent 交"总结: 详见原文。"六个字, 满分通过。

防法有两层, 且要一起上:

- ① **把校验器做硬**: 关键事实逐条机器可查 (实验 3 里偷懒者 4 项全挂); **但别死抠字面**——实验 3 的意外发现: 诚实 agent 把"耳朵疼"写成"耳痛", 被字面匹配的严格校验器误杀。好校验器 = 查实质, 不查台词。
- ② **审计校验器本身**: 对齐已学的 [[Critic泄漏审查]] (抓"针对评分集作弊"的六条拒绝规则) 和 [[LLM裁判与自动评估]] (裁判模型自己也会被钻)。

评估要评四样 (Q8), 其中"校验器"那一格, 本身就是被攻击面

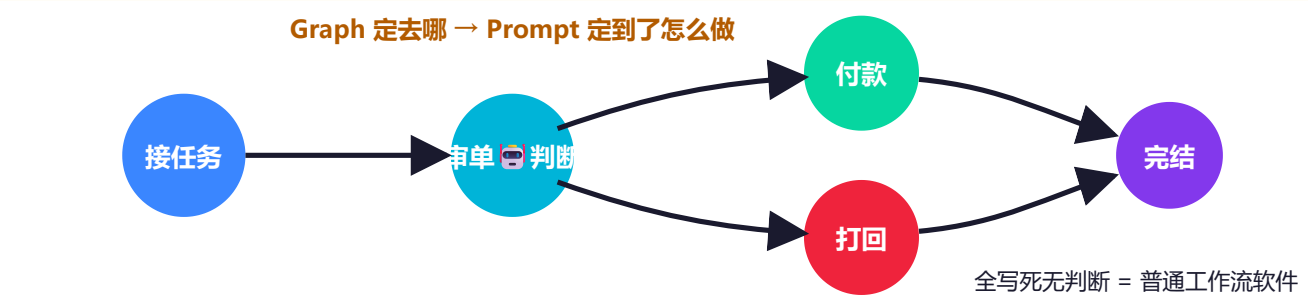


Q10 Graph 工程：Graph 和 Prompt 怎么分工？"全写死的 graph 只是 workflow 软件"怎么理解？

Graph 工程决定 **模型的下一步去哪里**（路由：审完→付款还是一打回？），Prompt 决定 **到了那里以后怎么做**（在那个节点怎么判断）。两者正交，缺一不可。

"全写死 = workflow 软件"：如果每个分支、每个判断都是硬编码规则，那 LLM 在里面只是个昂贵的文本格式化器——不如直接用 RPA/workflow 引擎。**Graph 里必须留给模型真正的判断点**，这个图才值得用 agent 跑。

🔗 已学对账：[[图环挽具三层工程]]（graph/loop/harness 三层 + 验证门控）正是 Graph 工程的体系化版本；[[确定性选择器模式]] 则是"哪些点不交给模型"的反面清单。两篇合读：Graph 层面先想清楚"哪里要判断、哪里必须死"。



Q11 "代码管边界，提示词管判断"——这条分界线具体怎么画？

这是全文的收束句，也是最好用的一条准绳。判据就一条：**这件事被搞错了，能不能靠"下次注意"挽回？**不能挽回的（钱出去了、文件删了、死循环了）归代码，能挽回的（质量好不好、路线优不优）归提示词。

归代码（硬边界）	归提示词（软判断）
最大轮数 / 超时	每轮怎么改进
文件与数据访问权限	用哪个工具、传什么参数
付款 / 发送等高危动作审批	报告写多细、风格怎么调
失败反馈的结构化封装	收到反馈后的策略调整

🔗 已学对账：[[沙箱与审批正交]] 讲"能不能 (sandbox) × 该不该 (approval)"两轴独立——正是这条分界线的落地机制；[[最小权限工具]] 思想同源。

Q12 五件套和已学的架构谱系是什么关系？真要造一个 agent，先动哪一件？

关系一句话：**架构谱系（Reflection/ReAct/多智能体...）回答“循环里跑什么模式”，五件套回答“让循环跑起来的工程面怎么铺”**——前者是阵型，后者是后勤。文章观点：这些都不新，是多年 agent 工作换了名字。

综合文章与已学的动工顺序建议：

① 先立 Eval（没有校验器，后面所有改动都不知道变好没变坏）→ ② Loop（校验器顺手给你失败反馈）→ ③ Context（每轮配菜）→ ④ Harness（工具与权限说明书）→ ⑤ Graph（流程大了才需要路由）

Eval 放第一的依据正是实验 1/3：**校验器既是评分官又是反馈源**——它定了“什么算对”，Loop 的反馈、Context 的取舍才有靶子。

📁 相关项目：**agent-architectures** (纲要75) 35 种架构怎么选 · **agent-harness** (ECC) 七职责怎么搭 · **cmu-agents** Harness→Eval→Training 作业链 · 本项目把散点收拢成**五件套全景图**。



第三部分 · 这套方法论给我什么

能力清单：学会五件套之后，能干什么

能力	具体用法（结合已学项目）
① 诊断卡死循环	自己写的 agent 反复重跑不收敛时，先查反馈是不是"失败了请重试"式 0 比特（实验 1 的 checklist）
② 上下文瘦身	用四操作给长会话体检：该 Select 什么、该 Compress 什么、什么要 Isolate 给子 agent
③ 给自己的自动化装校验器	任何"AI 干活"脚本，先写严格校验器（实质逐条可查、不死抠字面），再谈别的
④ 识别假开源课	"闲鱼 49.9"式营销的甄别路径：顺链找本体→查是否免费原文→查有无仓库
⑤ 评审他人的 agent 方案	五件套当 checklist：边界在代码里吗？反馈带信息吗？校验器骗得动吗？graph 有判断点吗？

第四部分 · 动手实验（3 个，全部零 API 实测跑通）

主实验 • exp1: 坏反馈 vs 好反馈 (Loop 工程)

步骤 1 | 搭环境: 任务模拟"读 data.csv 求和写 report.md", agent 只有 read_csv / write_file 两个工具, 模型用规则策略模拟 (确定性、零 API)。

```
cd F:\reminder\agent-engineering\exercise
set PYTHONUTF8=1 && python exp1_loop_feedback.py
```

步骤 2 | 看坏反馈组: 校验器只回"失败了, 请重试" (反馈内容被压扁成 0 比特)。

✅ 预期: 5 轮全部"原样再写一遍占位报告", 任务失败。

实测: **round1~round5 逐字重复同一动作, 5 轮全烧完 X**

步骤 3 | 看好反馈组: 同一策略, 改收结构化反馈 {"step": "compute", "missing": "报告里没有 TOTAL", "fix": "读 csv 算 sales 列总和"}。

✅ 预期: 按 fix 行动后通过。

实测: **round2 读 csv 算出 TOTAL: 800, 校验通过 ✓ (2 轮收敛)**

步骤 4 | 改一个参数看变化: 把 validate() 里好反馈的 fix 字段清空再跑——退化回坏反馈组行为。反馈的信息量这一个变量, 决定收敛与否。

实验 · exp2: 四操作救注意力 (Context 工程)

步骤 1 | 构造被污染的历史: 11 条消息 = 系统提示 + 6 条 v1 旧文档 + 切换通知 + 2 条 v2 新文档 + 收尾指令。

```
python exp2_context_ops.py
```

步骤 2 | 全量喂 (不处理)

实测: v1 出现 21 次 vs v2 出现 4 次 → 关键词策略选中 `fetch_data_v1` (已废弃版本) X——旧信息以数量优势骗过注意力, token 681。

步骤 3 | 四操作处理后再喂: Select 只留切换后消息, Compress 把 6 条旧文档折成一行"fetch_data_v1 即将下线, 新代码禁用"摘要, Isolate 隔离细节。

实测: token 681→265 (38%) • v1:3 vs v2:4 → 选中 v2 ✓——省 token 与提准度是同一件事。

踩坑实录: 首跑把 Select 切分点写在索引 6 (应为 7), 最后一条 v1 文档漏进上下文, **四操作组照样选错 v1, 实验结论整体翻车**——Select 的切分点选错一根线, 全盘皆输, 恰好反证"该给模型看什么"必须精确。

实验 · exp3：白卷拿满分（Eval 工程钻空子演示）

步骤 1 | 四象限对照：{诚实, 偷懒} 两 agent × {糊涂, 严格} 两校验器。偷懒 agent 输出固定六个字："总结：详见原文。"

```
python exp3_eval_gaming.py
```

步骤 2 | 读四行结果

实测：**偷懒×糊涂 = ☒ 满分通过（白卷 100 分）**；偷懒×严格 = 4 项事实全挂现行；诚实×糊涂 = 通过但纯属侥幸（糊涂校验器什么都放行）。

步骤 3 | 意外发现（比预期更有价值）：诚实 agent 的摘要写了"佩戴**耳痛**"，被死查字面"耳朵疼"的严格校验器**误杀**。

教训：校验器要查实质、别查台词——过字面化的严格，同样会惩罚诚实者。评估标准 = 机器可查 + 语义容错。

一页总结：五件套口袋卡

件	一句话	口诀	对应实验
Context	每轮给模型看什么	Write记 · Select挑 · Compress压 · Isolate隔	exp2
Harness	包住模型的工具/技能/权限/运行时	说明书也是提示词	—（已学七职责）
Loop	失败反馈驱动收敛	0比特反馈=原样重跑；反馈带 step/missing/fix	exp1
Eval	自报完成≠完成	评任务/环境/校验器/轨迹；校验器松一寸钻一尺	exp3
Graph	路由与判断分工	Graph定去哪 · Prompt定怎么做；全写死=工作流软件	—（已学三层工程）

总结一句话：**代码管边界，提示词管判断。**

产出：F:\reminder\agent-engineering\ | 指南 PDF + exercise\ 三个实验脚本（全部实测） | task f0affeb6-1b20-453b-ad29-87cd3c336cf8 | 来源：x.com/miles_mazy/status/2100955147786686575 → 长文 2100515498840805759 | 知识库互链：[[AgentHarness智能体挽具]] [[Compaction上下文压缩算法]] [[上下文预算与

